2-TIER CLOUD-BASED STUDENT MANAGEMENT SYSTEM USING TERRAFORM & FLASK (AWS)

---

1. Overview

This project demonstrates the automated deployment of a scalable **Student Management System** using a **2-tier architecture on AWS**, provisioned entirely through **Infrastructure as Code (IaC) using Terraform**.

The application layer is developed using **Python Flask (Gunicorn)**, and the database layer is managed using **Amazon RDS (MySQL)**.

---

2. Objective

- Automate infrastructure provisioning using Terraform
- Implement a 2-tier architecture (Application + Database)
- Deploy a Flask Web API application on AWS cloud
- Enable full CRUD operations for student data
- Ensure secure communication between application and database

---

3. Architecture Overview

Flow:

User → EC2 (Flask API) → Amazon RDS (MySQL Database)

Components:

- Application Layer → Hosted on AWS EC2 instance
- Database Layer → Amazon RDS MySQL

---

4. Technologies Used

- Python Flask → Backend framework
- Gunicorn → WSGI server
- PyMySQL → MySQL connectivity
- Terraform → Infrastructure provisioning
- AWS EC2 → Application hosting
- AWS RDS (MySQL) → Managed database
- Amazon Linux 2023 / Ubuntu → Server OS
- REST API → Client-server communication

---

5. Infrastructure Provisioned Using Terraform

LALITHA PRASAD

a. Network Layer

- Virtual Private Cloud (VPC)
- Public Subnet (EC2)
- Private Subnet (RDS)
- Internet Gateway
- Route Tables

---

b. Compute Layer

- EC2 Instance for Flask application
- User Data script for automated setup

---

c. Database Layer

- Amazon RDS MySQL instance
- Database: studentdb
- Secure subnet placement

---

6. Security Groups Configuration

Application Server (EC2)

Inbound Rules:

- SSH (22) → Admin access
- HTTP/Custom (5000) → Flask API access

Outbound Rules:

- Allow all traffic (for RDS connectivity)

Database Server (RDS)

Inbound Rules:

- MySQL (3306) → Only from EC2 Security Group

Security Highlights

- Database not publicly accessible
- Access restricted only to application layer
- Controlled network traffic using Security Groups

LALITHA PRASAD

7. Security Best Practices Implemented

- Least privilege access model
- Private subnet for database layer
- Security group-based access control
- No direct internet access to RDS
- Secure credential handling via Terraform variables

---

8. System Design

Application Layer

- Developed using Flask REST API
- Handles CRUD operations
- Uses PyMySQL for database interaction

Database Layer

- MySQL hosted in Amazon RDS
- Stores student records securely

---

9. Implementation Steps

1. Defined infrastructure using Terraform
2. Created VPC, subnets, and networking resources
3. Provisioned EC2 instance with user_data script
4. Created RDS MySQL database
5. Configured security groups for secure access
6. Installed Flask and dependencies on EC2
7. Configured systemd/Gunicorn service
8. Deployed application and tested APIs

---

10. Application Execution (Automated via Terraform)

Environment Setup (User Data Script)

- Install Python3 and pip
- Install Flask, Gunicorn, PyMySQL
- Configure environment variables

Run Application

LALITHA PRASAD

```
gunicorn -w 2 -b 0.0.0.0:5000 app:app
```

Access Application

http://<EC2-PUBLIC-IP>:5000/api/students

---

11. Application Features

- Add new students
- View all students
- View student by ID
- Update student details
- Delete student records

---

12. API Endpoints

Method	Endpoint	Description
GET	/api/students	Get all students
GET	/api/students/{id}	Get student
POST	/api/students	Add student
PUT	/api/students/{id}	Update student
DELETE	/api/students/{id}	Delete student

---

13. Challenges Faced

- EC2 user_data script failures
- Package compatibility issues in Amazon Linux
- Flask runtime configuration issues
- RDS connectivity and security group issues
- Terraform provisioning debugging

---

14. Solutions Implemented

- Used Terraform templatefile() for automation
- Fixed EC2 package installation issues
- Configured proper security group rules
- Enabled correct MySQL inbound rules

LALITHA PRASAD

- Added logs for debugging cloud-init issues

---

15. Outcome

- Successfully deployed infrastructure using Terraform
- Fully automated application deployment
- Secure 2-tier AWS architecture achieved
- Application accessible via public endpoint
- End-to-end cloud automation completed

---

16. Key Advantages of Terraform-Based Deployment

- Infrastructure automation
- Repeatable deployments
- Version-controlled infrastructure
- Reduced manual errors
- Faster provisioning in AWS

---

17. Conclusion

This project demonstrates real-world Cloud and DevOps practices by combining:

- Infrastructure as Code (Terraform)
- AWS Cloud Architecture
- 2-tier application design
- Backend development using Flask API

It provides strong hands-on experience in:

- Cloud Engineering
- Infrastructure Automation
- DevOps fundamentals
- Secure AWS architecture design

LALITHA PRASAD